

Problem 1: Denoising the Egyptian Goose with PCA and NMF

MATH70102 Unsupervised Learning — Coursework 1

1 Problem and approach

We are given a clean and a noisy 512×512 greyscale image of an Egyptian goose, as pixel matrices X_{clean} and X_{noisy} . For a selection of integers q covering $1, \dots, 512$ we reconstruct the noisy image in two ways and measure how close each reconstruction is to the clean image in *Euclidean (Frobenius) distance*

$$d(A, B) = \sqrt{\sum_{i,j} (A_{ij} - B_{ij})^2}.$$

PCA (Week 2): treating the 512 rows as samples in $\mathbb{R}^{512}$, we keep the first q principal components and reconstruct. **NMF** (Week 3): we factorise $X_{\text{noisy}} \approx WH$ with $W \in \mathbb{R}_{\geq 0}^{512 \times q}$, $H \in \mathbb{R}_{\geq 0}^{q \times 512}$ and reconstruct via WH . Writing $d_0 = d(X_{\text{clean}}, X_{\text{noisy}})$ for the baseline, a method *denoises* if some q gives $d(X_{\text{clean}}, \hat{X}_q) < d_0$.

We load the data and define the distance. The noisy image is already strictly positive, so it is directly admissible to NMF without the +1.01 shift used in the Week 3 digit example (whose data lived in $[-1, 1]$ with zeros).

```
import matplotlib.pyplot as plt
import numpy as np
import numpy.typing as npt
from sklearn.decomposition import PCA

X_clean = np.loadtxt("egyptian_goose_clean.csv", delimiter=",")
X_noisy = np.loadtxt("egyptian_goose_noisy.csv", delimiter=",")

def euclid(A, B):
    # Euclidean / Frobenius distance
    return np.sqrt(np.sum((A - B) ** 2))

baseline = euclid(X_clean, X_noisy) # d_0 = 29.5459
```

2 Choosing the grid of q via a scree plot

We let the data guide the grid using a scree plot of the noisy image's principal components — the Week 2 tool for deciding how many components to retain. The variance is heavily front-loaded (PC1 alone $\approx 27\%$, PC2 $\approx 14\%$; 80% is reached at $q = 42$, 95% at $q = 188$), so we sample densely at low q , refine over the mid-band where most variance is captured, and keep a coarse tail to $q = 512$ (included as a check: there the reconstruction must equal the noisy image).

```
scree = PCA().fit(X_noisy)
evr = scree.explained_variance_ratio_
cum = np.cumsum(evr)

fig, axs = plt.subplots(1, 2, figsize=(8, 2.7))
axs[0].bar(range(1, 31), evr[:30] * 100, color="salmon")
axs[0].set_xlabel("Principal component"); axs[0].set_ylabel("Explained variance (%)")
axs[1].plot(range(1, len(cum) + 1), cum * 100, color="royalblue")
axs[1].set_xlabel("Number of components q"); axs[1].set_ylabel("Cumulative (%)")

q_grid = np.array(sorted(set(
    list(range(1, 21))
    + [25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 90, 100]
    + [110, 125, 150, 175, 200, 225, 250, 275, 300]
    + [350, 400, 450, 500, 512])))
```

3 PCA reconstruction (Week 2)

For each q we fit `PCA(n_components=q)`, project with `fit_transform`, and reconstruct with `inverse_transform`. PCA re-adds the sample mean, so at $q = 512$ the noisy image is recovered exactly.

```
pca_dist = np.empty(len(q_grid)); pca_recons = {}
for i, q in enumerate(q_grid):
    pca = PCA(n_components=int(q))
    recon = pca.inverse_transform(pca.fit_transform(X_noisy))
    pca_dist[i] = euclid(X_clean, recon)
    pca_recons[int(q)] = recon
```

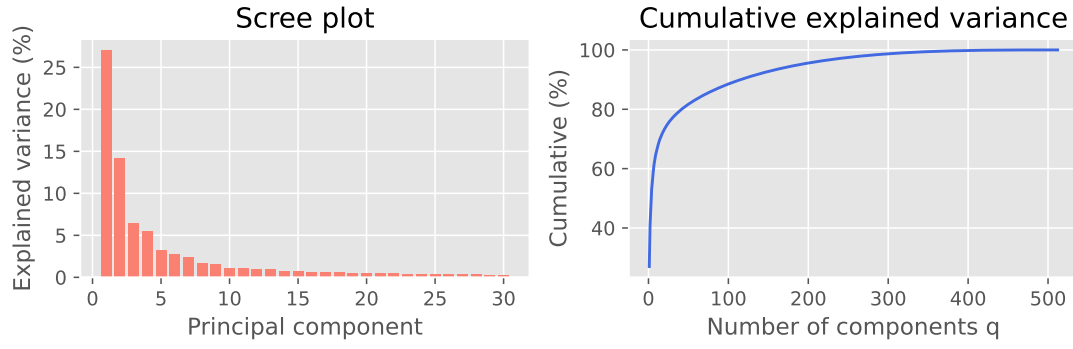


Figure 1: Scree plot (left) and cumulative explained variance (right) of the noisy image. The steep decay justifies dense sampling at low q .

```
best_pca_idx = int(np.argmax(pca_dist)); best_pca_q = int(q_grid[best_pca_idx])
# PCA: best q = 80 -> d = 24.9536; d at q=512 = 29.5459 (= baseline)
```

4 NMF reconstruction (Week 3)

We use the `NMF_numpy` class from the Week 3 reference *verbatim*. It maximises the quasi-likelihood $L(W, H) = \sum X \odot \log(WH) - WH$ by multiplicative updates; the reconstruction is stored in `.WH`.

```
class NMF_numpy:
    def train(self, X: npt.NDArray[np.number], q: int, n_iter: int = 100) -> None:
        d, n = X.shape
        self.W = np.random.uniform(low=0, high=1, size=(d, q)).astype(X.dtype)
        self.H = np.random.uniform(low=0, high=1 / q, size=(q, n)).astype(X.dtype)
        self.WH = self.W @ self.H
        self.logL = np.empty(n_iter, dtype=X.dtype)
        self.dist = np.empty(n_iter, dtype=X.dtype)
        for a in range(n_iter):
            self.W *= ((X / self.WH) @ self.H.T) / self.H.sum(axis=1)[None, :]
            self.WH = self.W @ self.H
            self.H *= (self.W.T @ (X / self.WH)) / self.W.sum(axis=0)[:, None]
            self.WH = self.W @ self.H
            self.logL[a] = np.sum(X * np.log(self.WH) - self.WH)
            self.dist[a] = np.sum((X - self.WH) ** 2)
```

A convergence check at $q = 20$ (Figure 2) shows the objective flattens well before 500 iterations, so we use `n_iter=400` in the sweep, with the random W, H initialisation seeded for reproducibility.

```
np.random.seed(0)
check = NMF_numpy(); check.train(X_noisy, q=20, n_iter=500)
fig, axs = plt.subplots(1, 2, figsize=(8, 2.4))
axs[0].plot(range(1, 501), check.logL, color="red"); axs[0].set_ylabel(r"$L(W, H)$")
axs[1].plot(range(1, 501), check.dist, color="blue"); axs[1].set_ylabel(r"$\|X - WH\|^2$")

np.random.seed(0)
nmf_dist = np.empty(len(q_grid)); nmf_recons = {}
for i, q in enumerate(q_grid):
    nmf = NMF_numpy(); nmf.train(X_noisy, int(q), n_iter=400)
    nmf_dist[i] = euclid(X_clean, nmf.WH)
    nmf_recons[int(q)] = nmf.WH

best_nmf_idx = int(np.argmax(nmf_dist)); best_nmf_q = int(q_grid[best_nmf_idx])
# NMF: best q = 275 -> d = 25.2179
```

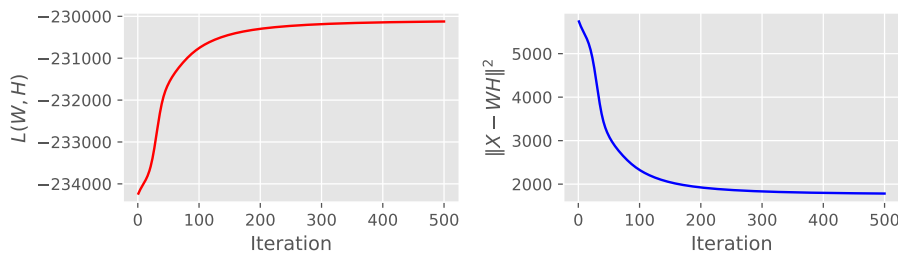


Figure 2: NMF convergence at $q = 20$: quasi-likelihood $L(W, H)$ (left) and reconstruction error $\|X - WH\|^2$ (right) against iteration. Both have plateaued by ~ 200 iterations.

5 Results

```
plt.plot(q_grid, pca_dist, "-o", label="PCA")
plt.plot(q_grid, nmf_dist, "-s", label="NMF")
plt.axhline(baseline, linestyle="--", label="Baseline = %.2f" % baseline)
plt.xlabel("q"); plt.ylabel("distance to clean image"); plt.legend()
```

	best q	$d(X_{\text{clean}}, \hat{X}_q)$	improvement vs. d_0	cum. variance at q	denoises?
Baseline (noisy)	—	29.55	—	—	—
PCA	80	24.95	4.59	86%	yes
NMF	275	25.22	4.33	98%	yes

Table 1: Best reconstructions. PCA beats the baseline for all $q \in [25, 500]$; NMF for all $q \in [35, 512]$.

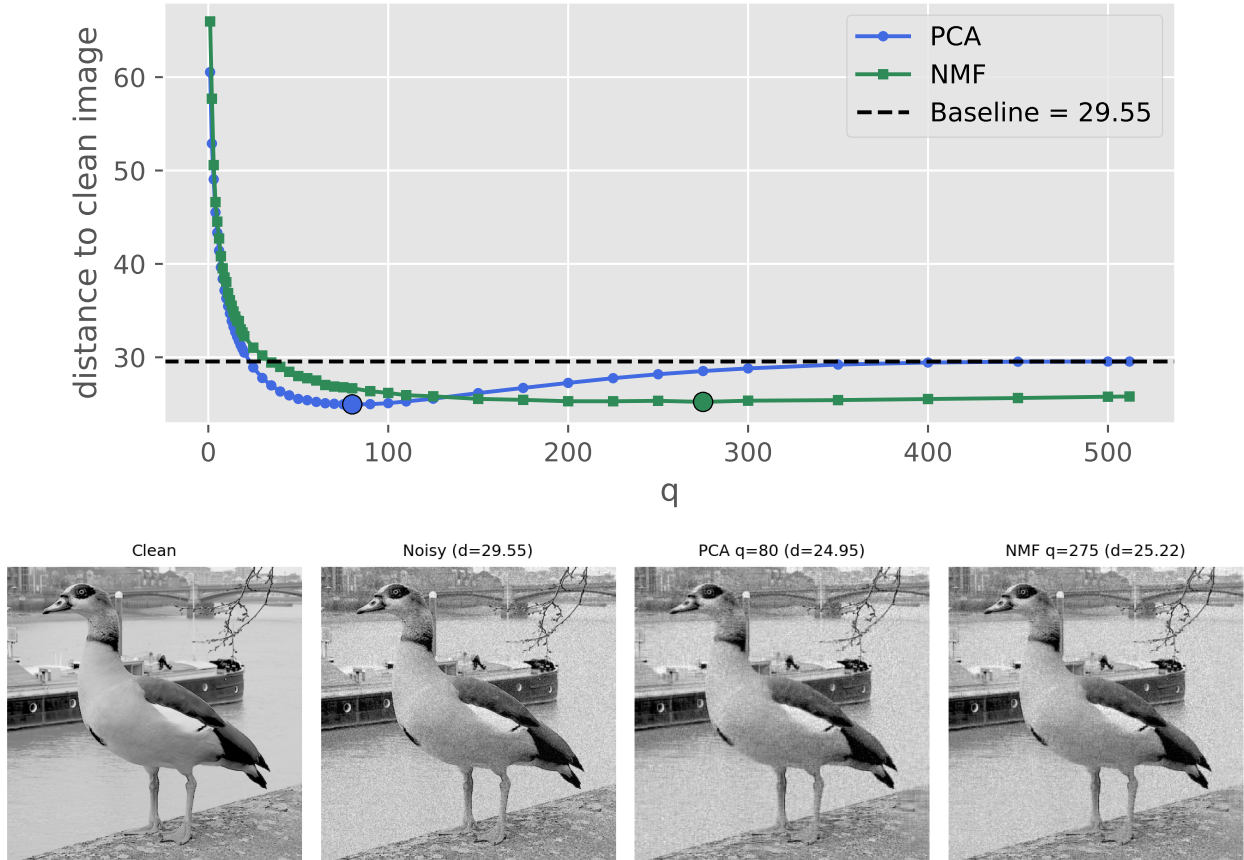


Figure 3: Top: distance to the clean image versus q ; both curves dip well below the baseline (dashed) and the marked points are the optima. Bottom: clean, noisy, and best- q PCA/NMF reconstructions (displayed on the clean image’s intensity range).

6 Conclusion

Yes — both PCA and NMF denoise the image. For each method there is a wide band of q giving a reconstruction strictly closer to the clean image than the noisy image is: the distance falls from $d_0 = 29.55$ to 24.95 (PCA, $q \approx 80$) and 25.22 (NMF, $q \approx 275$). The distance-versus- q curve is **U-shaped**. Noise spreads its energy across all 512 directions, whereas the image’s structure is concentrated in the leading components (scree plot: $PC1 \approx 27\%$). For small q the reconstruction under-fits and discards genuine structure; at an intermediate q it keeps the signal while rejecting the many noise-dominated high-index components — the denoising optimum; and as $q \rightarrow 512$ it recovers the noisy image ever more faithfully, so the distance climbs back to d_0 (for PCA this is exact, verified at $q = 512$). PCA reaches its optimum earlier and slightly lower than NMF, consistent with its components being the variance-optimal directions, so it concentrates the signal into fewer of them. The improvement is real but moderate ($\approx 15\%$): broadband noise can only be partly removed by a low-rank truncation, since only the portion of the noise lying in the discarded components is rejected.